JANMITRA

Civic Issue Reporting & Management System

Team ResolveIT | 4 Categories | 4 Wards | 4 Departments

Team ResolveIT

Project Breakdown & Architecture Guide

Complete project structure, modules, tasks and responsibilities

Team Members & Their Colour Code in This Document

Anas Shaikh	Backend Core + Infrastructure Lead
Anurag Deshmukh	AI Module + Dashboard Backend
Om Gadekar	Frontend Lead + Officer/Worker Portal
Faizan Rayeen	Citizen Portal + Notifications

40-Day Development Timeline Overview

Phase 0	Days 01-03	Setup & Environment
Phase 1	Days 04-10	Foundation: Auth, Models, APIs
Phase 2	Days 11-20	Core Workflow: AI, Assignment, Worker, Citizen
Phase 3	Days 21-28	Department Dashboard
Phase 4	Days 29-36	Testing, Bugfix & Deployment
Phase 5	Days 37-40	Demo Prep & Finalisation

1. Project Summary

Project Name	JANMITRA -- Civic Issue Reporting & Management System
Team	ResolveIT
College	B.Tech Final Year Project
Problem	Citizens have no digital way to report civic issues to NMC and track their resolution.
Solution	A web app where citizens report issues, AI validates the photo, system routes to the correct NMC department and ward, workers fix it, citizens verify.
Categories	4 only: Roads & Infrastructure | Garbage & Waste | Water & Drainage | Street Lighting
Wards	4 only: Ward 1, Ward 2, Ward 3, Ward 4
Departments	PWD | Solid Waste Management | Public Health Engineering | Electrical Department
Organisation	Nagpur Municipal Corporation (NMC)
Total Days	40 working days
Cost	Rs. 0 -- all free tools and hosting

2. System Flow -- End to End

Step 1	Citizen opens JANMITRA in phone browser. No app download needed.
Step 2	Citizen registers with: Name, Mobile, Ward (1-4), Password.
Step 3	Citizen submits a complaint: selects 1 of 4 categories, selects ward, writes description, uploads 1-3 photos, confirms GPS location on map.
Step 4	System saves complaint and returns Complaint ID immediately. Status = Submitted.
Step 5	In background, Celery worker runs AI validation on the photo: - Layer 1: Checks if photo is fake (EXIF metadata + Error Level Analysis + HuggingFace AI detector) - Layer 2: Checks if photo shows a real civic issue (YOLOv8 object detection) - Also: Checks if same issue already reported within 100m in same ward (duplicate detection)
Step 6	If photo is fake: Status = Rejected. Citizen notified. Must resubmit with real photo.
Step 7	If photo is real: Priority score calculated. Complaint auto-routed to correct department. Status = Routed.
Step 8	Dept. Officer sees complaint in their dashboard. Assigns a worker. Worker must be from same department AND same ward as the complaint.
Step 9	Worker receives push notification. Sees task in their list. Task shows: complaint photos, description, GPS location on map.
Step 10	Worker goes to location, fixes the issue, uploads proof photos, marks task as Work Done.
Step 11	Citizen receives notification: 'Work completed. Please verify.' Sees proof photos.
Step 12	Citizen accepts: Complaint Closed + 5-star rating. Citizen rejects: Complaint Reopened. No response in 72 hours: Auto-closed.
Step 13	Department Dashboard updates in real-time. Officer sees resolution stats, ward breakdown, worker performance.

3. All Modules Breakdown

Module 1: Authentication & Roles

Users	Custom User model: id (UUID), name, mobile (login), password, role, ward, department, designation, employee_id, status, FCM token
Roles	4 roles: citizen, worker, dept_officer, admin -- stored in user.role field
JWT Tokens	Access token (60 min) + Refresh token (7 days) via DRF SimpleJWT
RBAC	IsCitizen, IsWorker, IsDeptOfficer, IsAdmin classes on every API endpoint
Registration	Citizen self-registers. Workers registered by admin/officer. Officers created by admin.

Module 2: Complaint Lifecycle

Category Master	4 categories seeded in DB at deployment. Cannot be changed by users.
Ward Master	4 wards seeded in DB. Cannot be changed by users.
Dept. Master	4 departments seeded. Each maps to 1 category.
Complaint Model	Fields: id, citizen, category, department, ward, description, photos, GPS, status, priority score, fake flags, duplicate flags, assigned worker, override flags, timestamps
Status Flow	Submitted -> Validated/Rejected -> Routed -> Assigned -> In-Progress -> Work Done -> Closed/Reopened
Images	complaint_images table: complaint_id, file path, type (complaint or proof-of-work)
History	status_history: every status change logged with who changed it, when, and a note

Module 3: AI Validation Pipeline

Celery Task	process_complaint_ai() fires automatically after complaint submission. User does not wait.
Layer 1 Step 1	EXIF check (Pillow): real camera photos have metadata tags (GPS, camera model, datetime). AI-generated images have none.
Layer 1 Step 2	Error Level Analysis (OpenCV+NumPy): AI images are too uniform when recompressed. Real photos show natural variation.
Layer 1 Step 3	HuggingFace fake detector: pre-trained AI model classifies image as real or artificial. Threshold: 0.70 confidence.
Layer 2	YOLOv8 (fine-tuned): checks if image contains a real civic scene. Confidence < 0.5 goes to manual review queue.
Duplicate Check	PostGIS ST_DWithin: same category + same ward + GPS within 100 metres = duplicate. Linked to parent complaint.
Priority Engine	Score = severity*0.30 + safety*0.25 + public_impact*0.20 + duplicate_count*0.15 + dept_capacity*0.10

Module 4: Department Dashboard

Access	Dept. Officers see ONLY their department. Admin sees all 4 departments.
KPI Cards	Total / Pending / In-Progress / Resolved / Rejected + Avg. Resolution Time
Ward Bar Chart	X=Ward 1-4, Y=complaint count, grouped by status (Recharts BarChart)
Severity Pie	Critical/High/Medium/Low distribution (Recharts PieChart)
Trend Line	Complaints submitted vs resolved per day/week/month (Recharts LineChart)
Worker Workload	Table: worker name, ward, active tasks, completed this week, avg resolution time
High Priority	Table: all Critical+High open complaints with days open, assigned worker
Filters	Date range, Ward, Status, Severity -- affect all widgets simultaneously
Validation Queue	AI low-confidence complaints for manual review: validate or reject

Module 5: Worker Management

Worker Fields

	id, employee_id, full_name, mobile, email, department, ward, designation, status, password, FCM token, registered_at
Ward Rule	Every worker has 1 department + 1 ward. Both required. Cannot be null.
16 Combos	4 departments x 4 wards = 16 department+ward combinations. Demo seeds 1 worker per combo = 16 demo workers.
Assignment	Officer selects from filtered list: only workers with matching dept AND ward shown.
Override	Admin can override ward rule. Requires reason text. Logged in status_history.
Worker View	Worker task list filtered: request.user.department + request.user.ward only.

Module 6: Notifications

Firebase FCM	Firebase Cloud Messaging for browser push notifications. Free, no limits for our scale.
FCM Token	Stored in user.fcm_token. Updated on each login if changed.
Triggers	Complaint submitted, Validated, Routed, Assigned, Work Done, Closed, Reopened, Rejected
Background	send_push_notification Celery task. Non-blocking.
Foreground	Firebase JS SDK: shows toast notification when app is open.
Background Push	firebase-messaging-sw.js service worker: shows notification when tab is closed.

4. Technology Stack Summary

Layer	Technology	Purpose
Frontend	React 18 + Tailwind CSS	All 4 portals (citizen, worker, officer, admin)
Frontend	React Router v6	Navigation + role-based route protection
Frontend	React Context API	Auth state (token, user, role) -- no Zustand
Frontend	Axios	HTTP client + JWT auto-refresh interceptor
Frontend	Leaflet + OpenStreetMap	Maps: GPS pin, complaint location
Frontend	Recharts	Dashboard: bar, pie, line charts
Frontend	Firebase JS SDK	Receive push notifications in browser
Backend	Python 3.11 + Django 4.2 LTS	All server logic, ORM, migrations
Backend	Django REST Framework 3.15	REST API endpoints
Backend	DRF SimpleJWT	JWT access+refresh token authentication
Backend	django-cors-headers	Allow React to call Django API
Backend	Gunicorn	Production server on Render
Database	PostgreSQL 15 + PostGIS 3.3	Primary DB + spatial 100m duplicate query
AI Layer 1	Pillow + OpenCV + Transformers	Fake image detection: EXIF + ELA + HuggingFace
AI Layer 2	Ultralytics YOLOv8	Civic issue relevance check
Background	Celery 5 + Redis 7	Async AI pipeline + notifications
Notifications	Firebase Admin SDK	Send push from Django backend
Deployment	Docker + Docker Compose	Local dev (Celery needs Linux)
Deployment	Render Free Tier	Cloud: web service + background worker + static site
Monitoring	UptimeRobot	Keeps Render free service awake every 10 min

5. Database Tables Summary

Table	Key Fields	Relations
users	id, name, mobile, role, ward_id, dept_id, designation, status	FK: ward, department
categories	id(1-4), name, department_id -- STATIC, seeded	FK: department
wards	id(1-4), name -- STATIC, seeded	--
departments	id(1-4), name, short_name, head_user_id -- STATIC	FK: head user
complaints	id(UUID), citizen, category, dept, ward, status, priority, GPS...	FK: all above + worker
complaint_images	id, complaint_id, file, type (complaint or proof)	FK: complaint
status_history	id, complaint_id, old_status, new_status, changed_by, note, time	FK: complaint, user
notifications	id, recipient_id, title, body, complaint_id, is_read, time	FK: user, complaint

6. User Roles & Permissions

Action	Citizen	Worker	Dept. Officer	Admin
Submit complaint	YES	no	no	no
View own complaints	YES	no	no	no
View own tasks (dept+ward)	no	YES	no	no
Upload proof photo	no	YES	no	no
Mark Work Done	no	YES	no	no
Accept/reject resolution	YES	no	no	no
Assign workers	no	no	YES	YES
View dept. dashboard	no	no	YES	YES
Validate AI queue manually	no	no	YES	YES
Register workers	no	no	OwnDept	YES
Override ward rule	no	no	no	YES
View all departments	no	no	no	YES
View all wards	no	no	no	YES